AWS VPC Peering

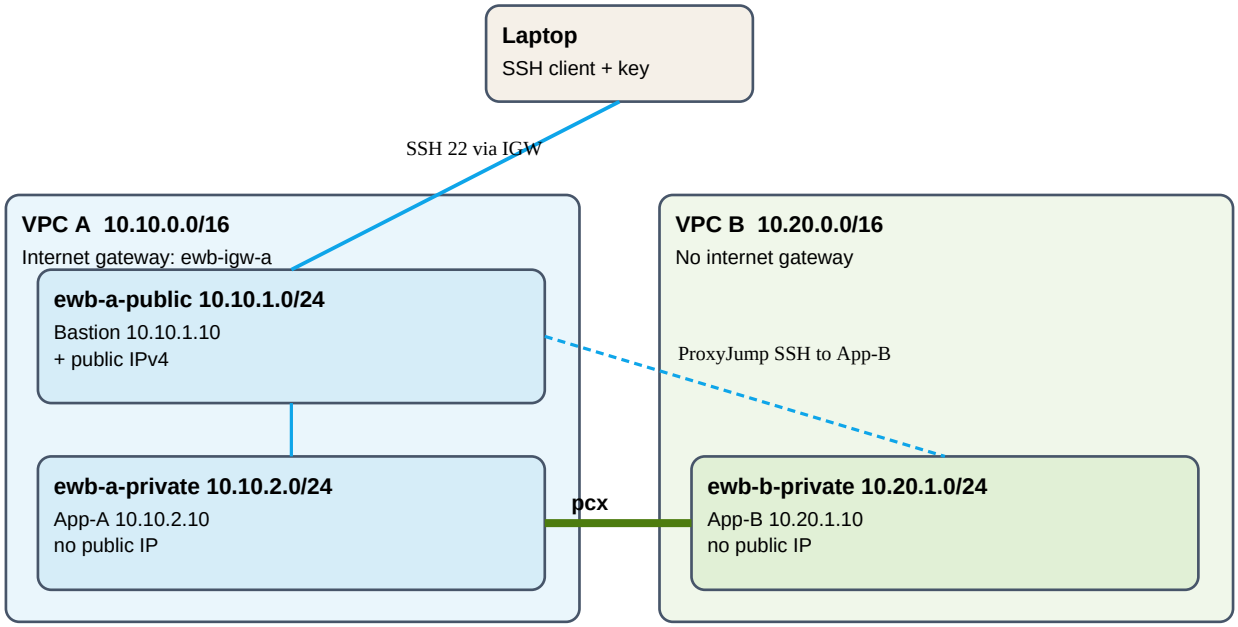
Connect two VPCs. Prove private connectivity.

Name	Shubham Sanjay Kalekar
Batch	AWS Internship Batch 1.1
Project	EWB Cloud Labs - Project 01: VPC Peering
Region	Asia Pacific (Mumbai) / ap-south-1
Availability Zone	ap-south-1a
Completion date	31 October 2026

This report documents a two-VPC peering lab: design, build, validation (including a failed baseline and a controlled failure) and the troubleshooting needed along the way. All screenshots are from my own execution. No private keys or passwords are included.

1. Architecture and inventory

Two isolated VPCs with non-overlapping CIDRs are joined by one peering connection. A single public bastion in VPC A is the only internet entry point. App-A and App-B have no public IPs and are reached with SSH ProxyJump through the bastion. VPC B has no internet gateway, and peering does not give it internet access through VPC A.



Administration path: laptop to bastion to App-A or App-B. Project traffic: App-A and App-B over private IPs through the peering connection.

Resource	Name	ID / value
VPC A	ewb-vpc-a	vpc-077ff559acdca39ef / 10.10.0.0/16
VPC B	ewb-vpc-b	vpc-00a942b237a518ef0 / 10.20.0.0/16
Subnet	ewb-a-public	subnet-030845f726b0eaa3f / 10.10.1.0/24
Subnet	ewb-a-private	subnet-0fbb1e8c9eecf8816 / 10.10.2.0/24
Subnet	ewb-b-private	subnet-0d0ff2a5f5074bbff / 10.20.1.0/24
Internet gateway	ewb-igw-a	igw-0d3b59b83ae82a17a (attached to ewb-vpc-a)
Route table	ewb-rt-a-public	rtb-0bb4f0c07cb564f32
Route table	ewb-rt-a-private	rtb-05f65bde9a575159e
Route table	ewb-rt-b-private	rtb-061f25df5bf9b6315
Security group	ewb-sg-bastion-VPC-A	sg-09f84599204424dd1
Security group	ewb-sg-app-a-VPC-A	sg-001914dbc3395234c
Security group	ewb-sg-app-b-VPC-B	sg-0b59569b2e3db6c42
Peering connection	ewb-a-b-peer	pcx-039fd11607154ad44
Instance	ewb-bastion	i-005da74d6fa8be696 / 10.10.1.10 / public 13.233.36.85
Instance	ewb-app-a	i-047b8a6e080de7be9 / 10.10.2.10 / no public IP
Instance	ewb-app-b	i-0ca39a73afc52117c / 10.20.1.10 / no public IP
Key pair	ewb-lab.pem	Stored on my laptop only; never copied to an instance

Note: the bastion was relaunched once because the first launch did not receive a public IPv4 address (auto-assign was not enabled in the launch network interface). The inventory shows the final bastion.

2. VPC, subnets and route tables

Both VPCs use non-overlapping /16 ranges, tagged Project = EWB-VPC-Peering and Student = my name. Each custom route table is explicitly associated with its subnet, so no subnet silently falls back to a main route table.

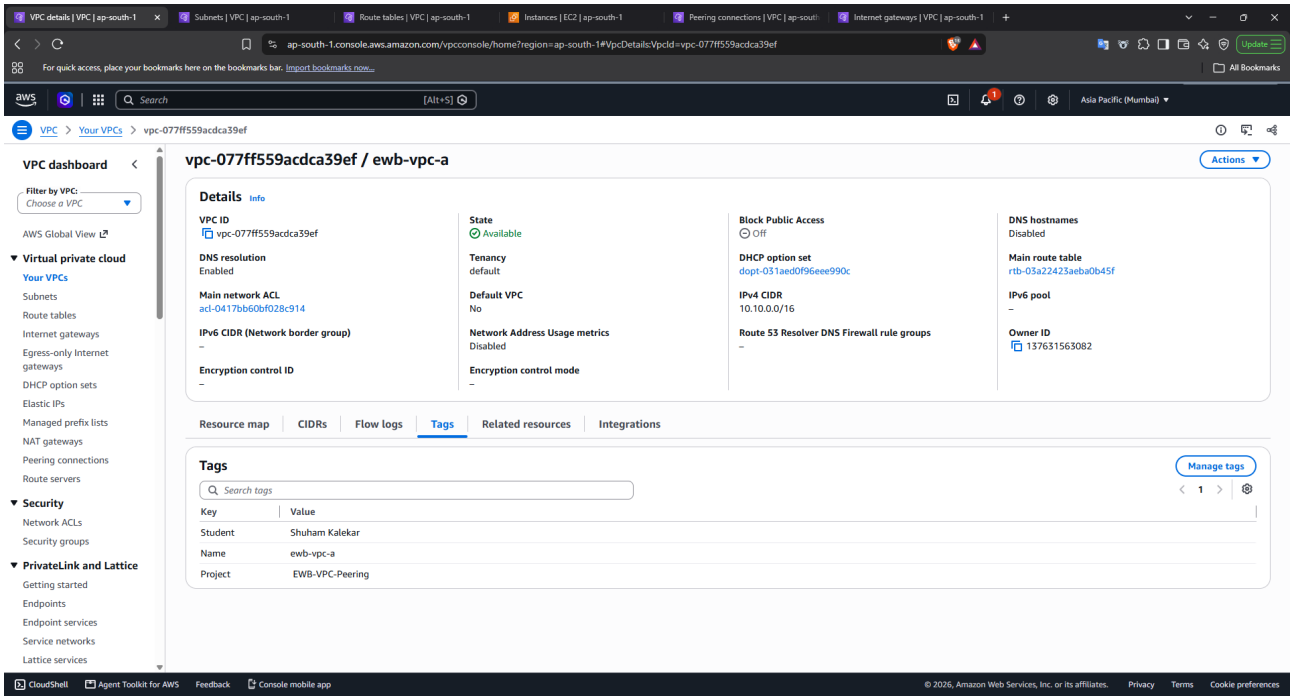


Figure 1 - ewb-vpc-a: 10.10.0.0/16, state Available, tags present.

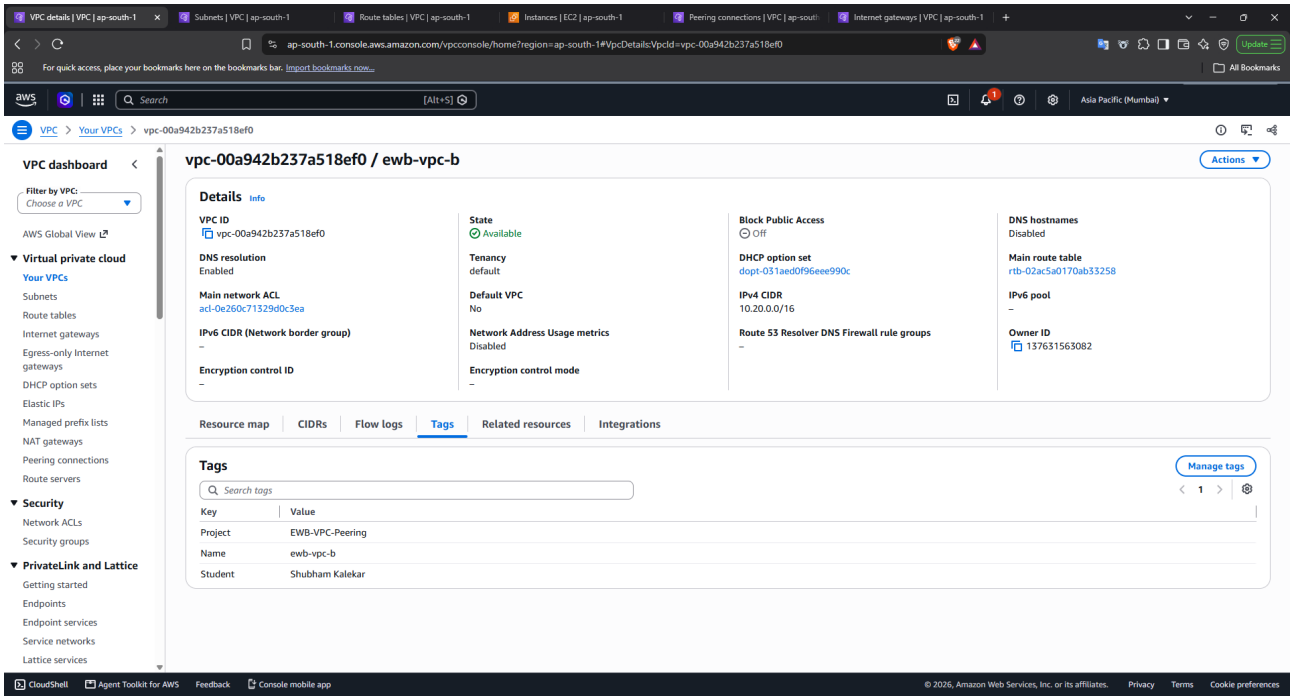


Figure 2 - ewb-vpc-b: 10.20.0.0/16, state Available, tags present.

Checkpoint 01 - route tables before peering

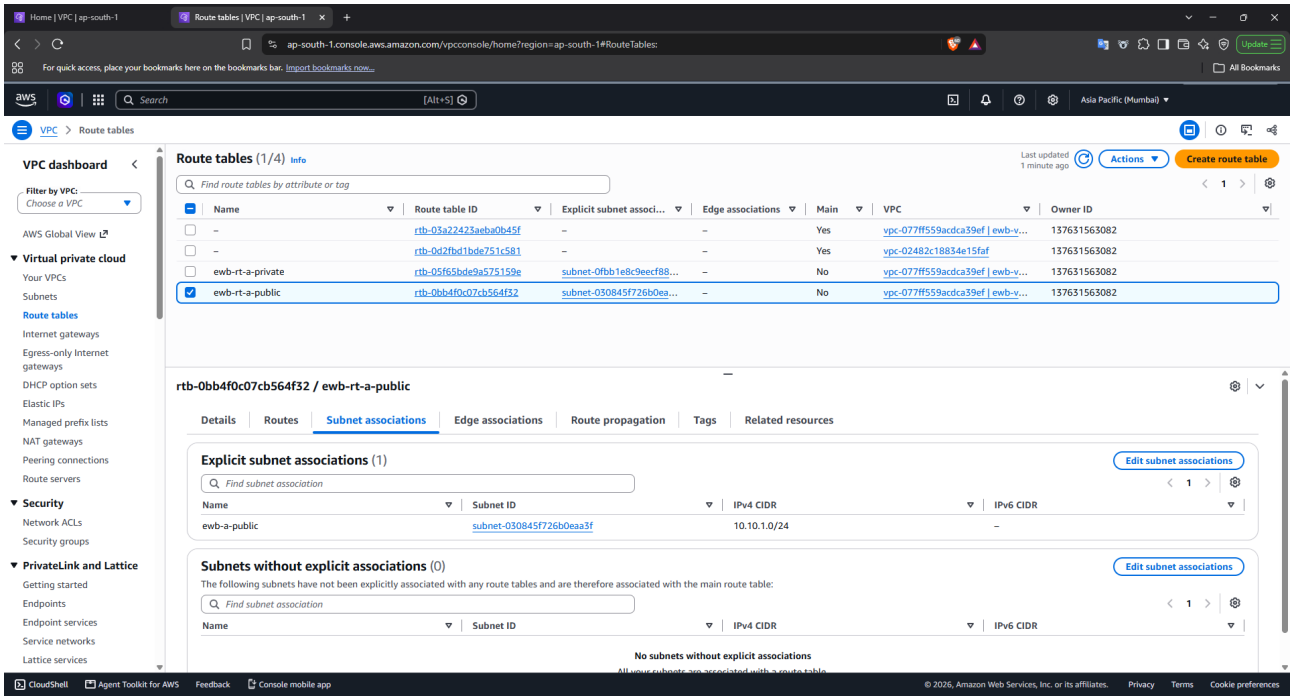


Figure 3 - ewb-rt-a-public: explicit association with ewb-a-public (10.10.1.0/24).

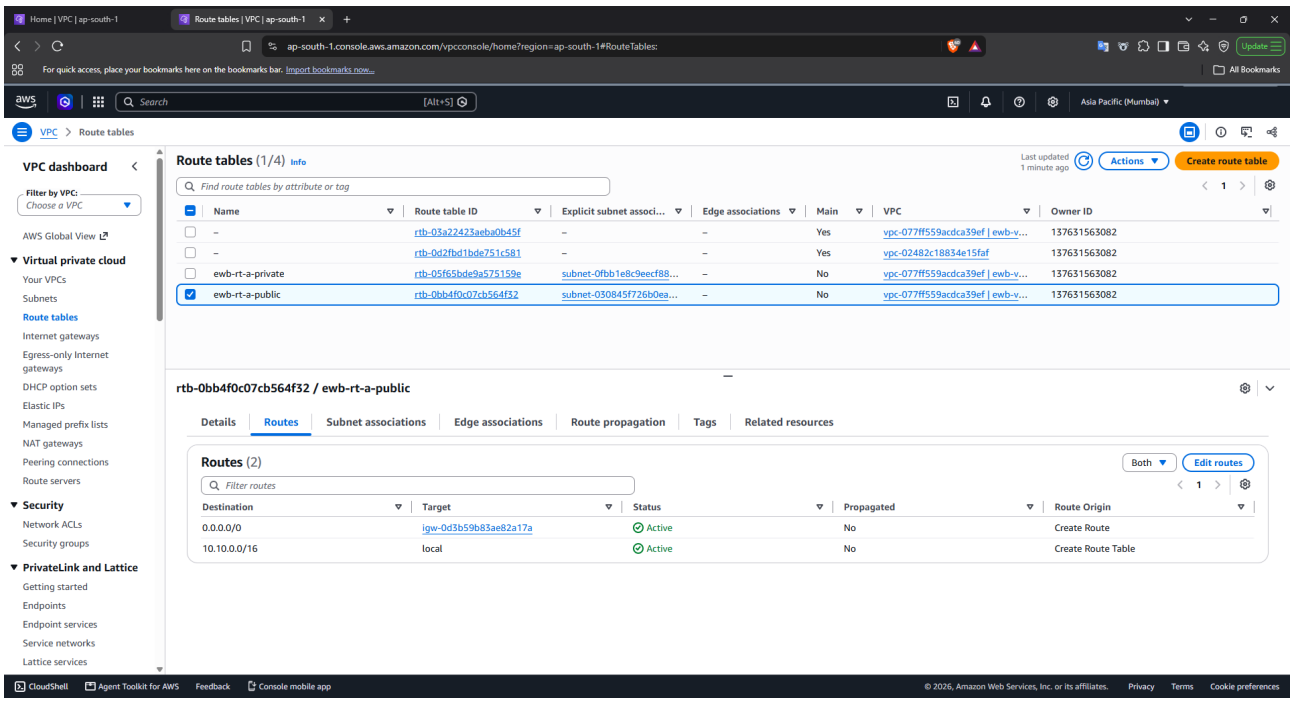


Figure 4 - ewb-rt-a-public routes before peering: 10.10.0.0/16 local and 0.0.0.0/0 to igw-0d3b59b83ae82a17a.

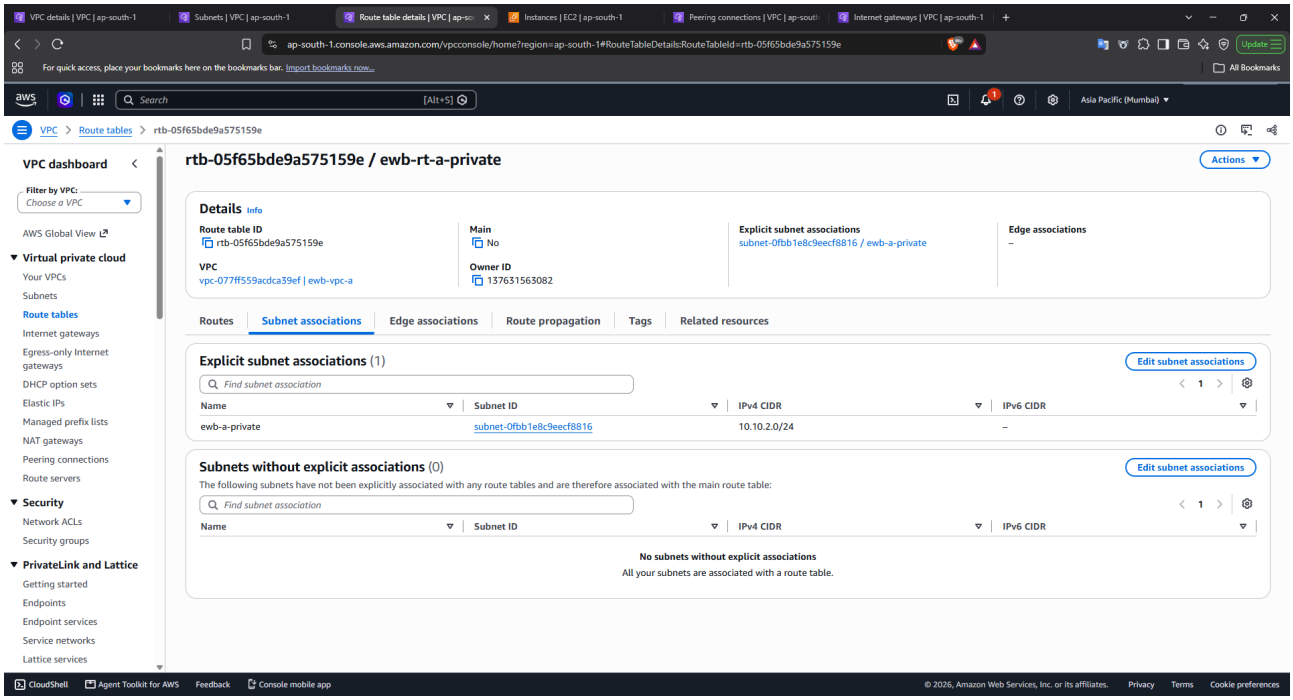


Figure 5 - ewb-rt-a-private: explicit association with ewb-a-private (10.10.2.0/24).

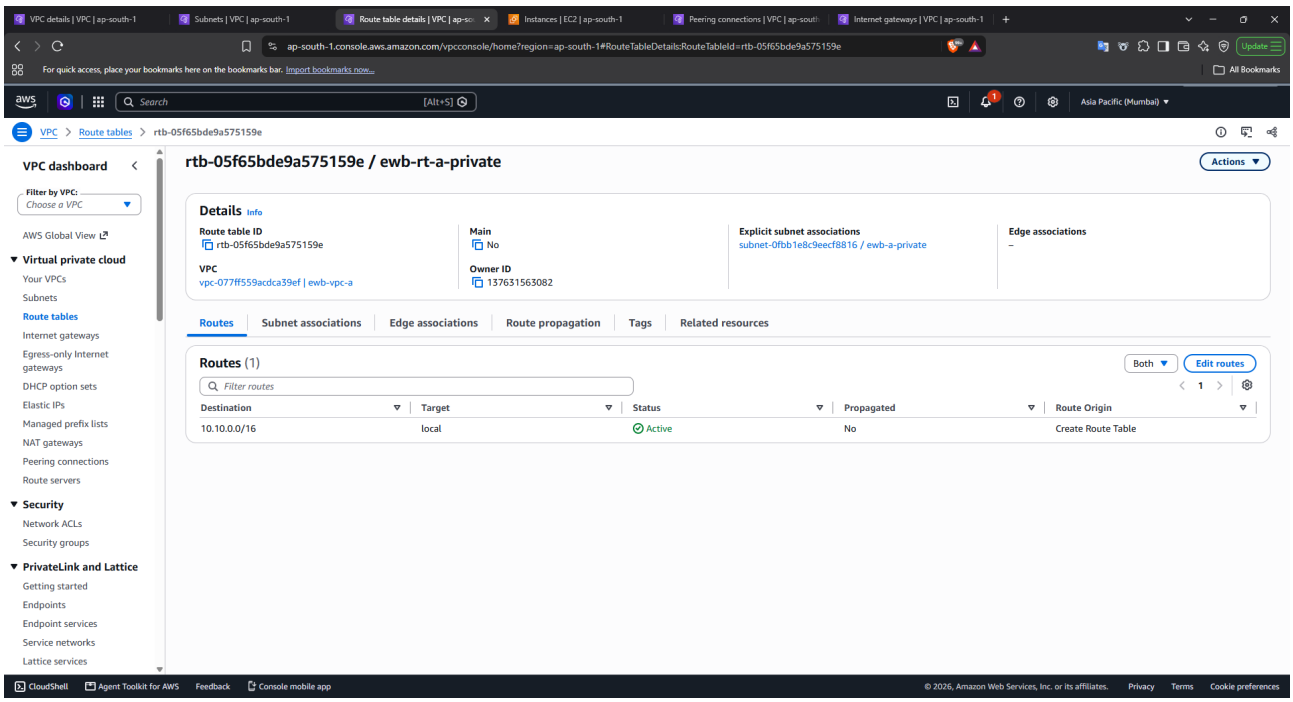


Figure 6 - ewb-rt-a-private before peering: local route only, no internet route.

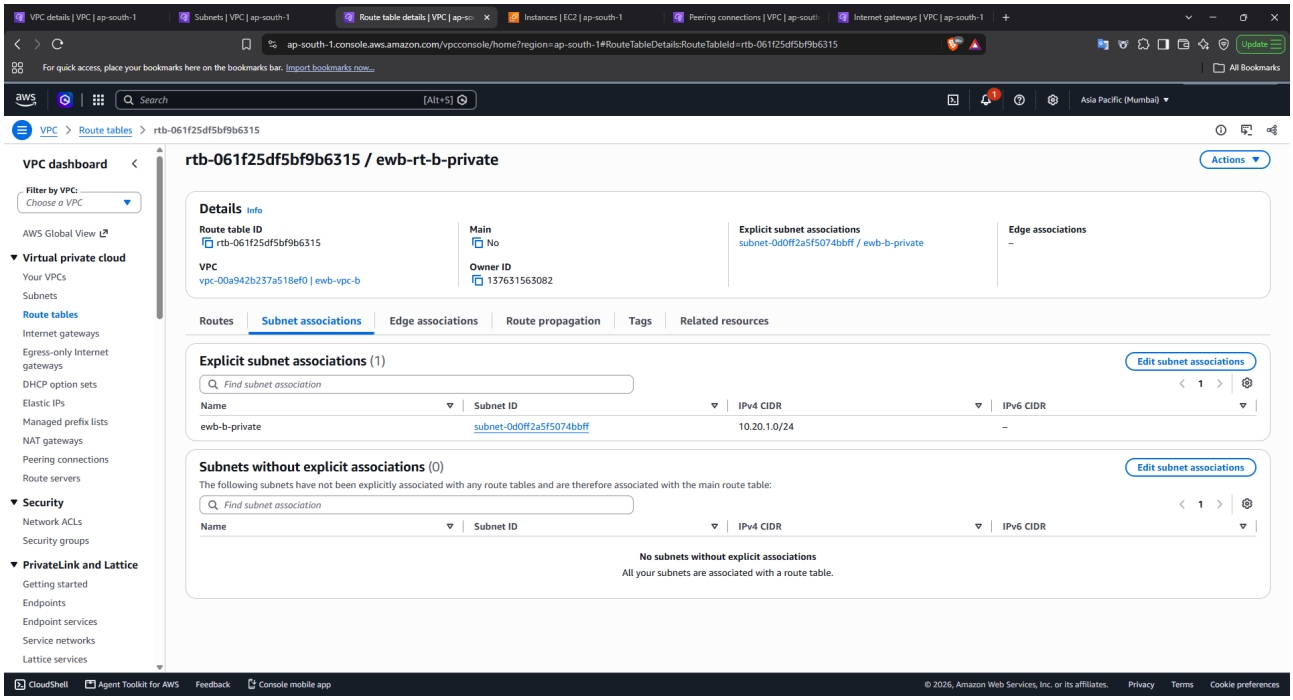


Figure 7 - ewb-rt-b-private: explicit association with ewb-b-private (10.20.1.0/24).

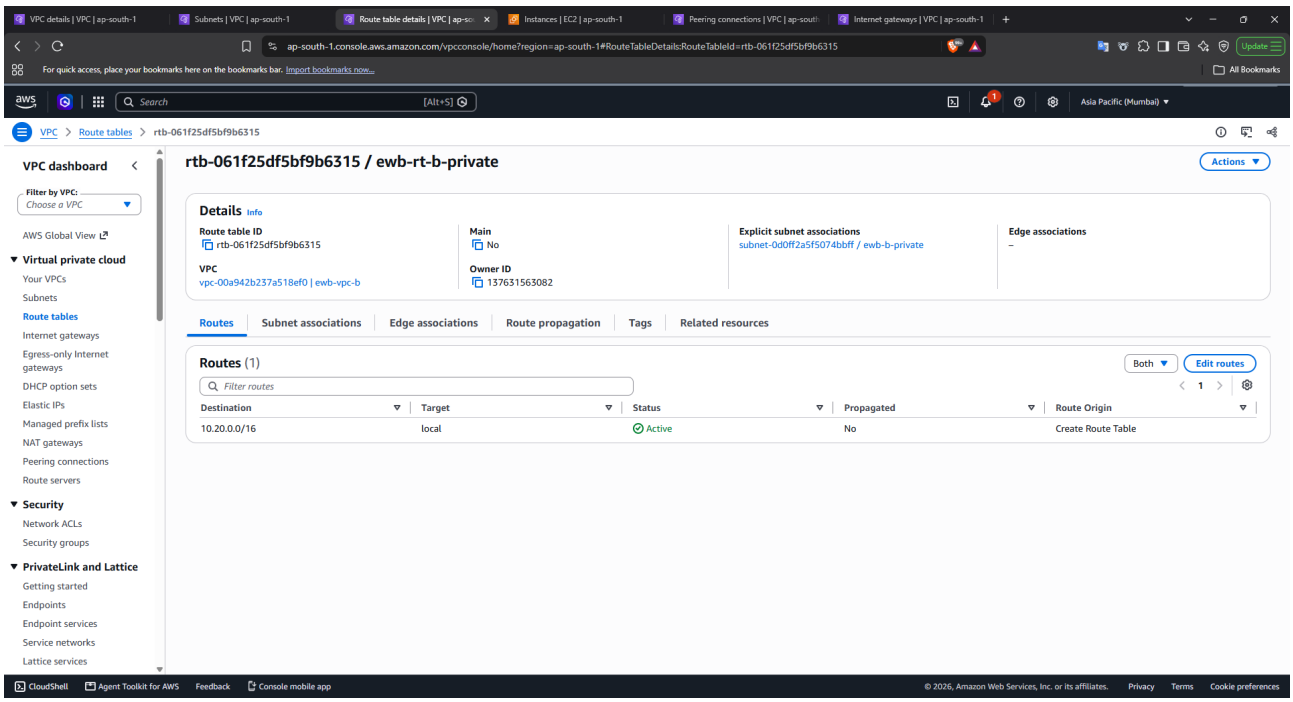


Figure 8 - ewb-rt-b-private before peering: local route only. VPC B has no internet gateway.

3. Instances and isolation

Three t3.micro Amazon Linux 2023 instances run in ap-south-1a. Only the bastion has a public IPv4 address; App-A and App-B show an empty Public IPv4 field, which satisfies the isolation requirement.

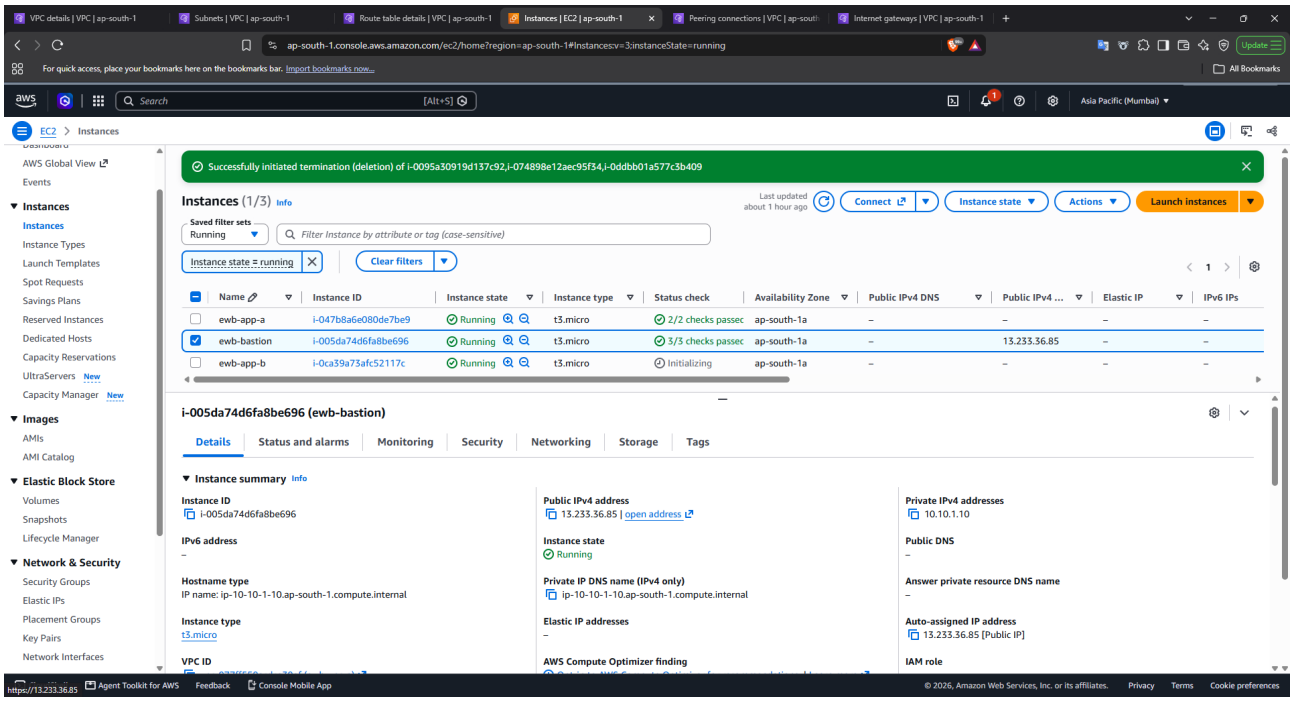


Figure 9 - ewb-bastion: private 10.10.1.10, public IPv4 13.233.36.85. The green banner is from terminating my earlier bastion that had no public IP.

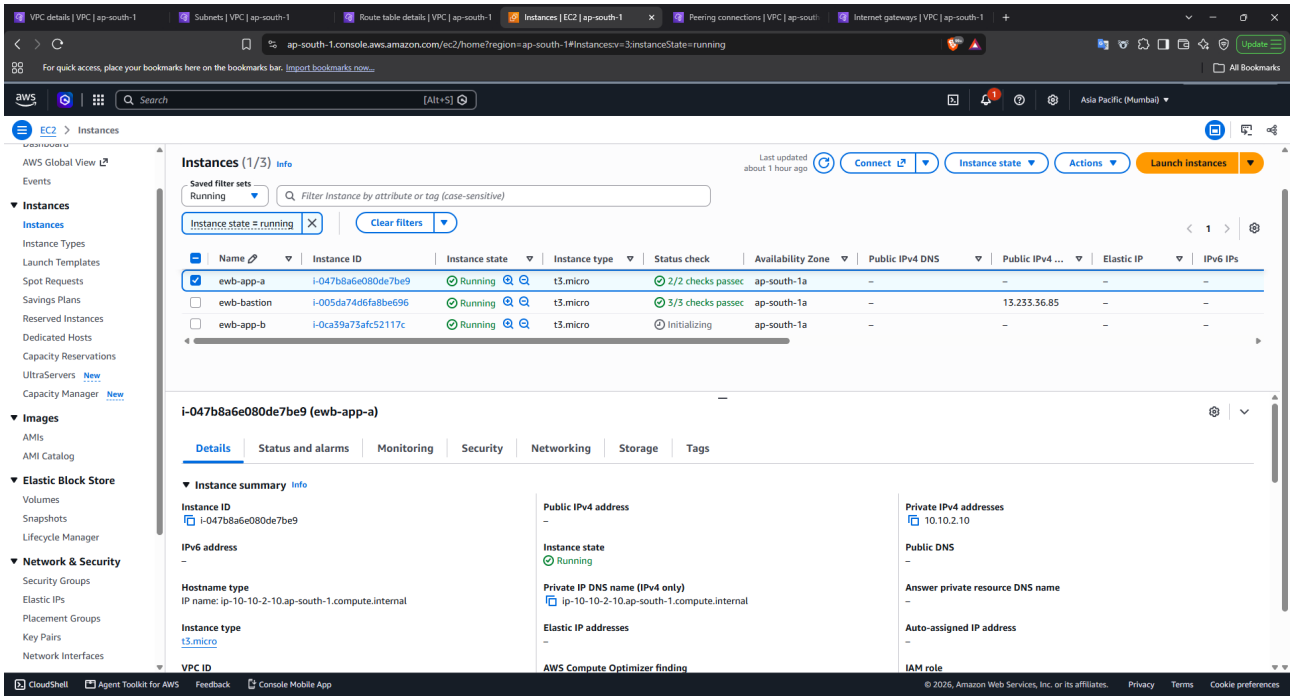


Figure 10 - ewb-app-a: private 10.10.2.10, Public IPv4 address empty, 2/2 checks passed.

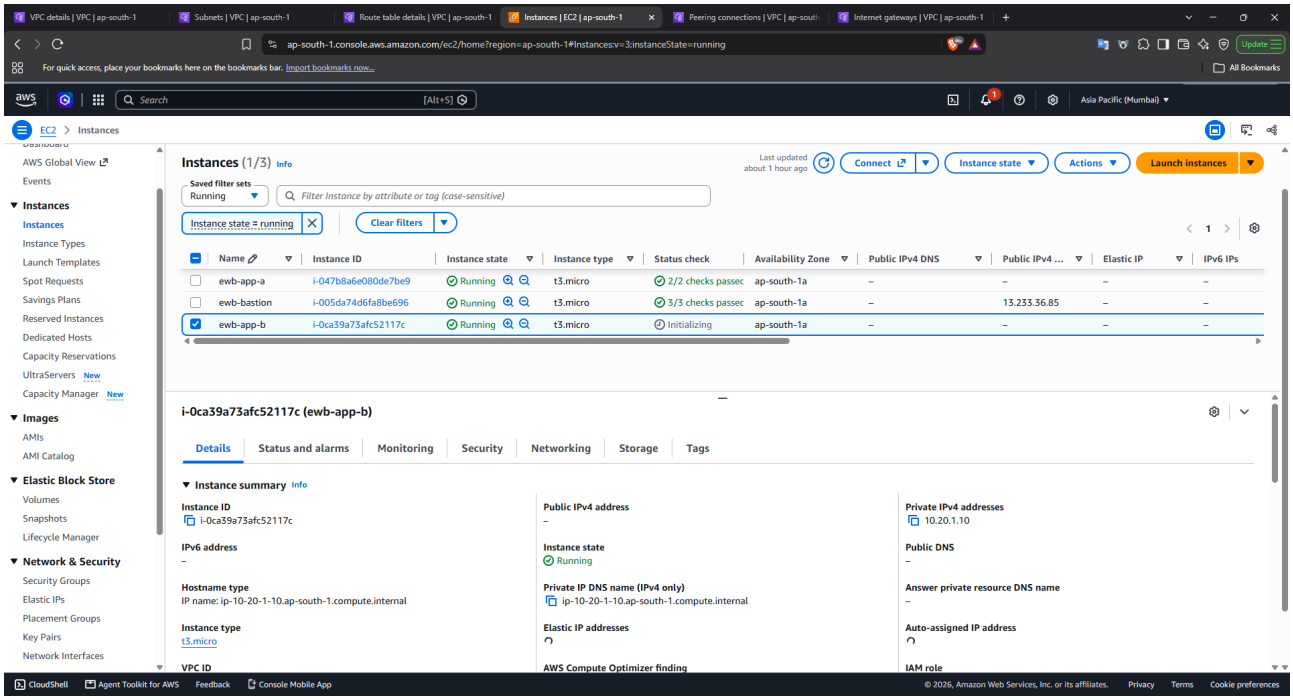


Figure 11 - ewb-app-b: private 10.20.1.10, Public IPv4 address empty. Status checks were still initializing when this was captured.

4. Security groups

One security group per instance. Intended inbound rules:

Security group	Rule	Source
ewb-sg-bastion-VPC-A	SSH / TCP 22	My current public IPv4 /32 (203.192.227.105/32)
ewb-sg-app-a-VPC-A	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-a-VPC-A	ICMP Echo Request (type 8)	10.20.1.10/32 (App-B)
ewb-sg-app-b-VPC-B	SSH / TCP 22	10.10.1.10/32 (bastion)
ewb-sg-app-b-VPC-B	ICMP Echo Request (type 8)	10.10.2.10/32 (App-A)

Outbound rules were left at the default allow-all. Security groups are stateful, so replies need no rule. Nothing is open to 0.0.0.0/0.

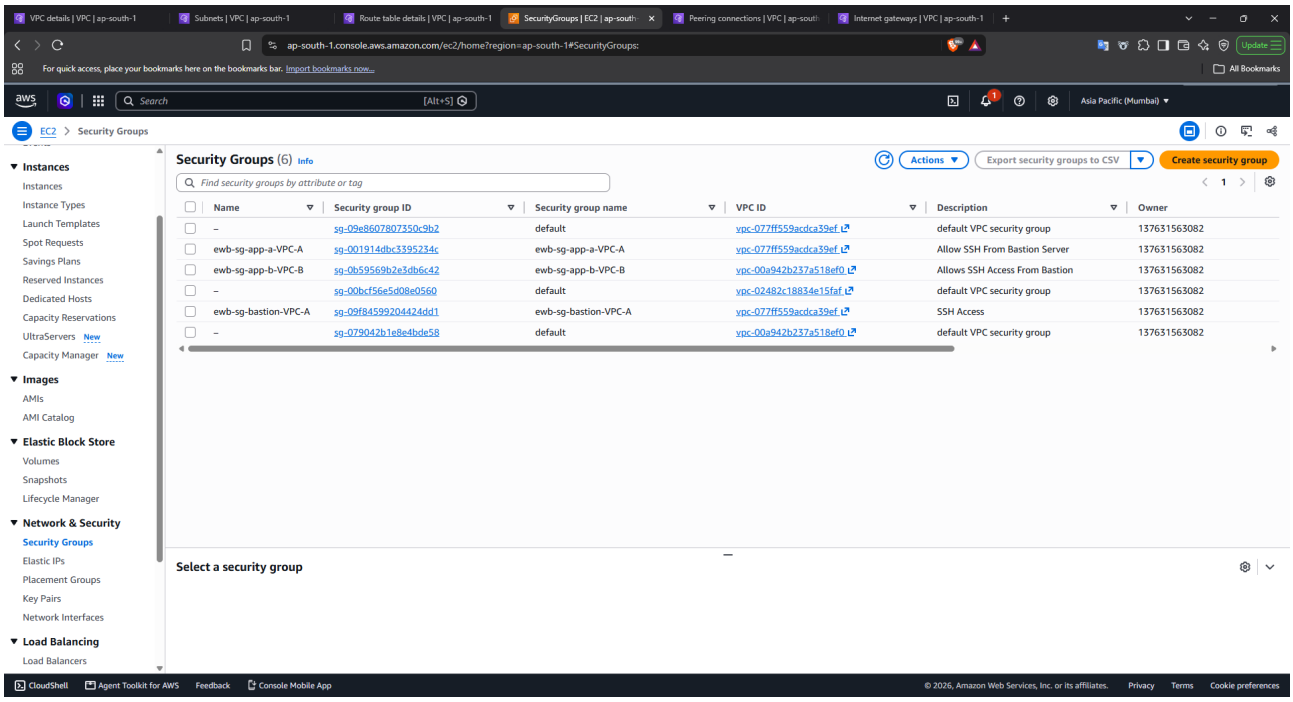


Figure 12 - Security group list: my three lab groups plus the three default VPC groups (one per VPC, created by AWS).

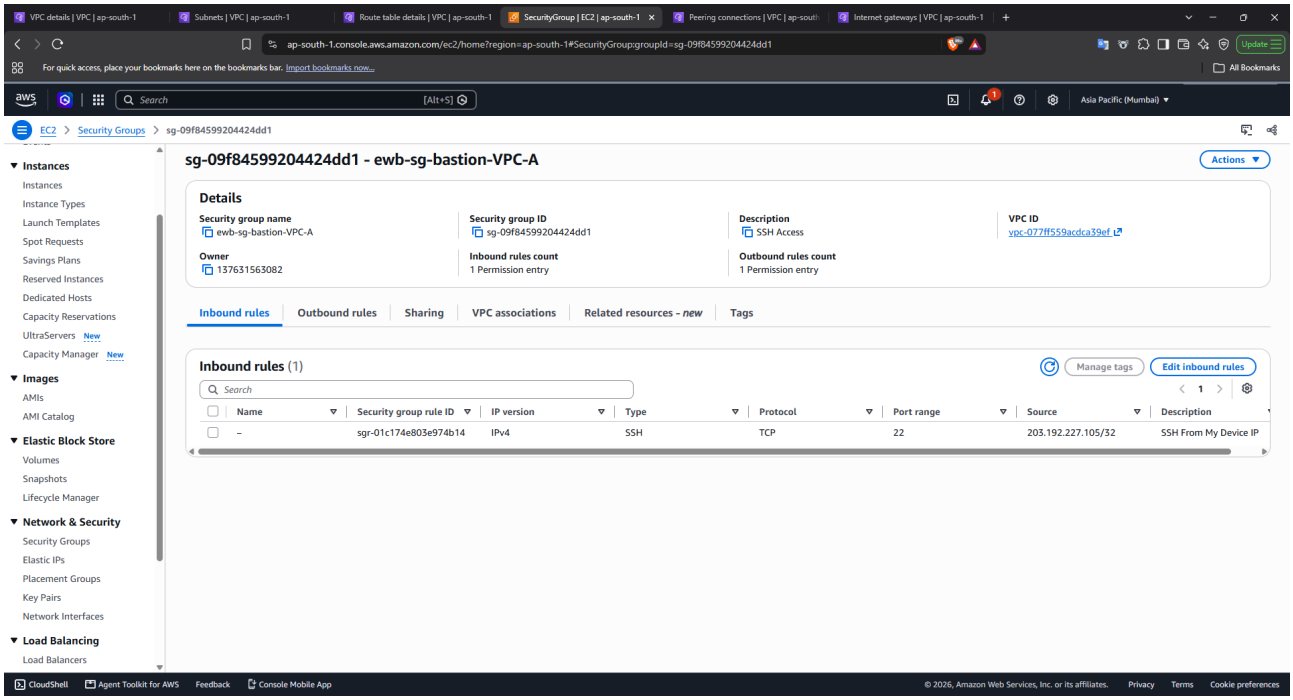


Figure 13 - ewb-sg-bastion-VPC-A: SSH 22 from a single /32 address.

5. SSH access through the bastion

The key stays on my Windows laptop. ~/.ssh/config defines ewb-bastion (public IP) and ewb-app-a / ewb-app-b with ProxyJump ewb-bastion, so the bastion only forwards the connection. The first attempt failed because the file was saved as config.txt; renaming it to config fixed it.

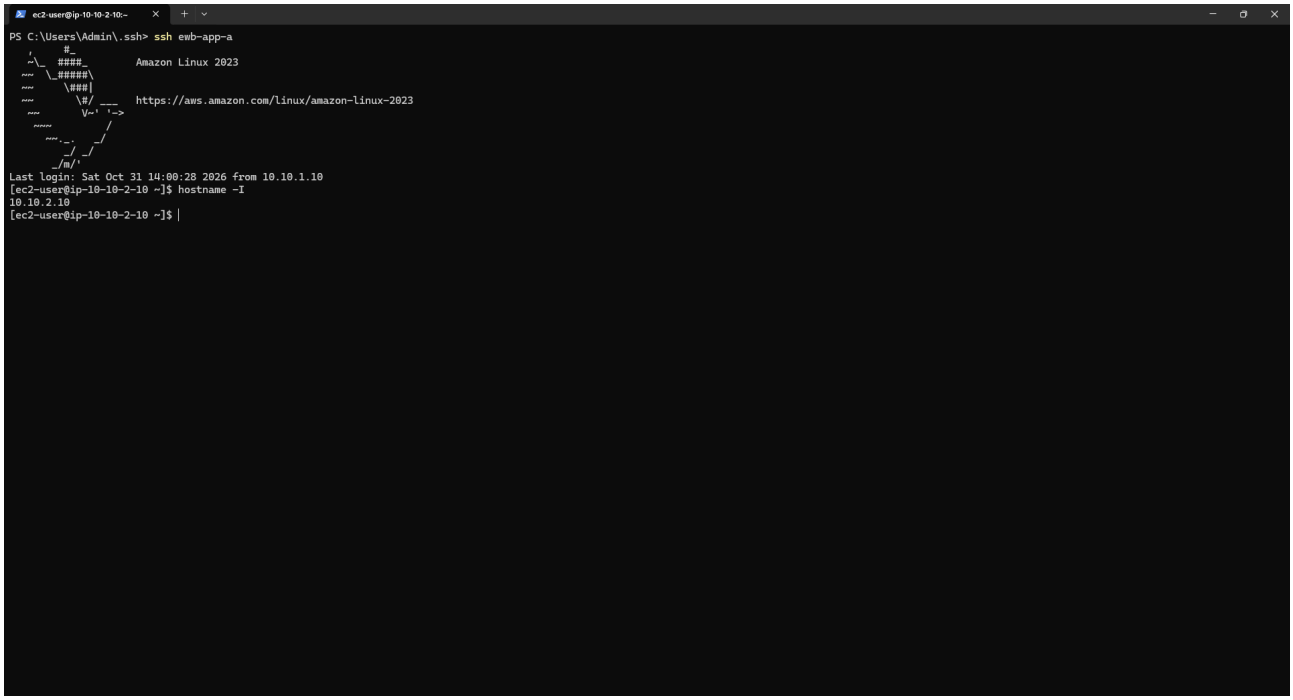


Figure 14 - ssh ewb-app-a through the bastion: hostname -I returns 10.10.2.10; Last login shows the source as 10.10.1.10 (the bastion).

6. Baseline: failed test before peering

From App-A, before peering and peer routes existed, VPC B is unreachable. Both VPCs are separate networks and App-A's route table has only its local route.

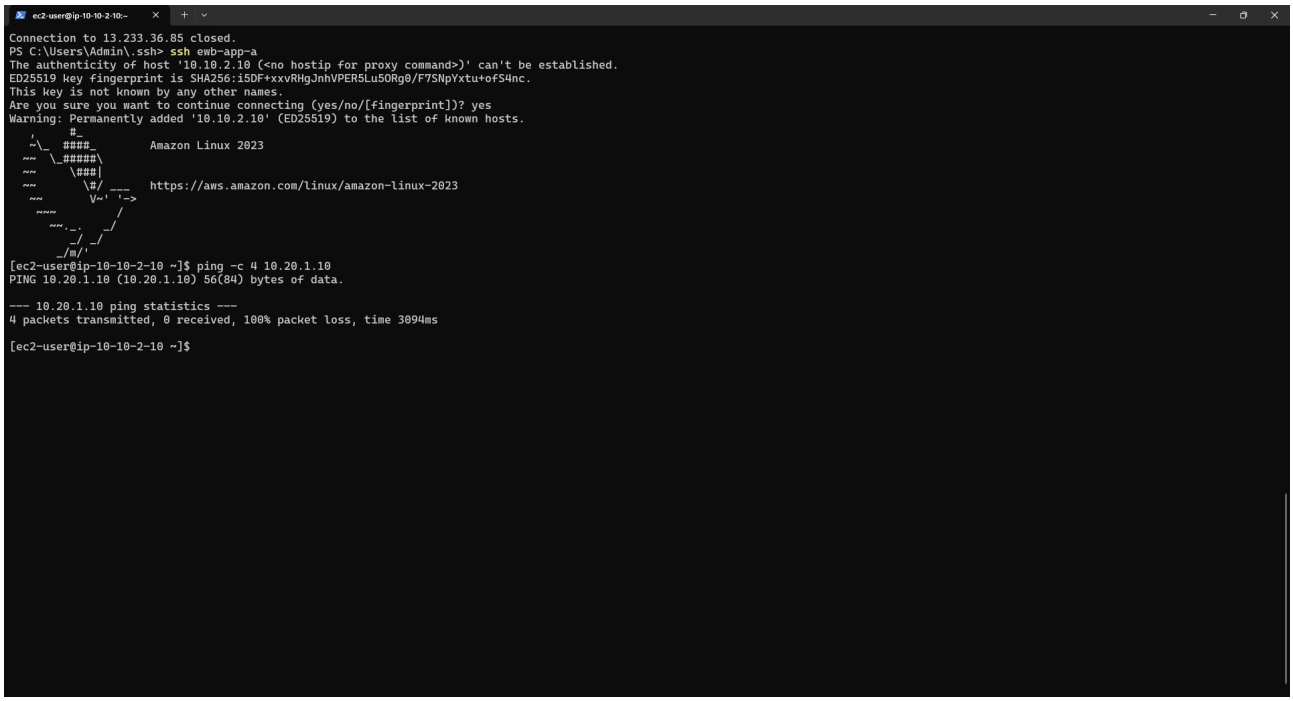


Figure 15 - Test 01: on ip-10-10-2-10, ping -c 4 10.20.1.10 gives 4 transmitted, 0 received, 100% packet loss.

7. VPC peering

ewb-a-b-peer was requested from ewb-vpc-a to ewb-vpc-b (same account, same Region) and had to be accepted manually even though both VPCs are mine.

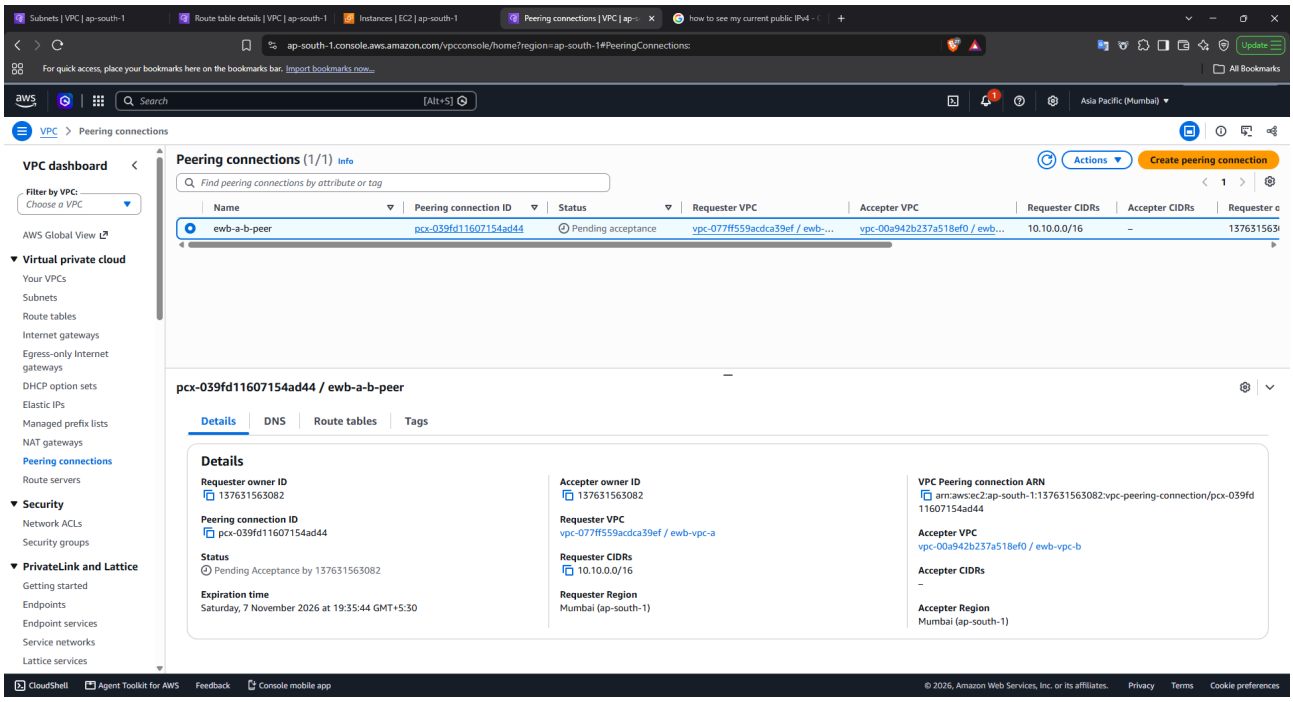
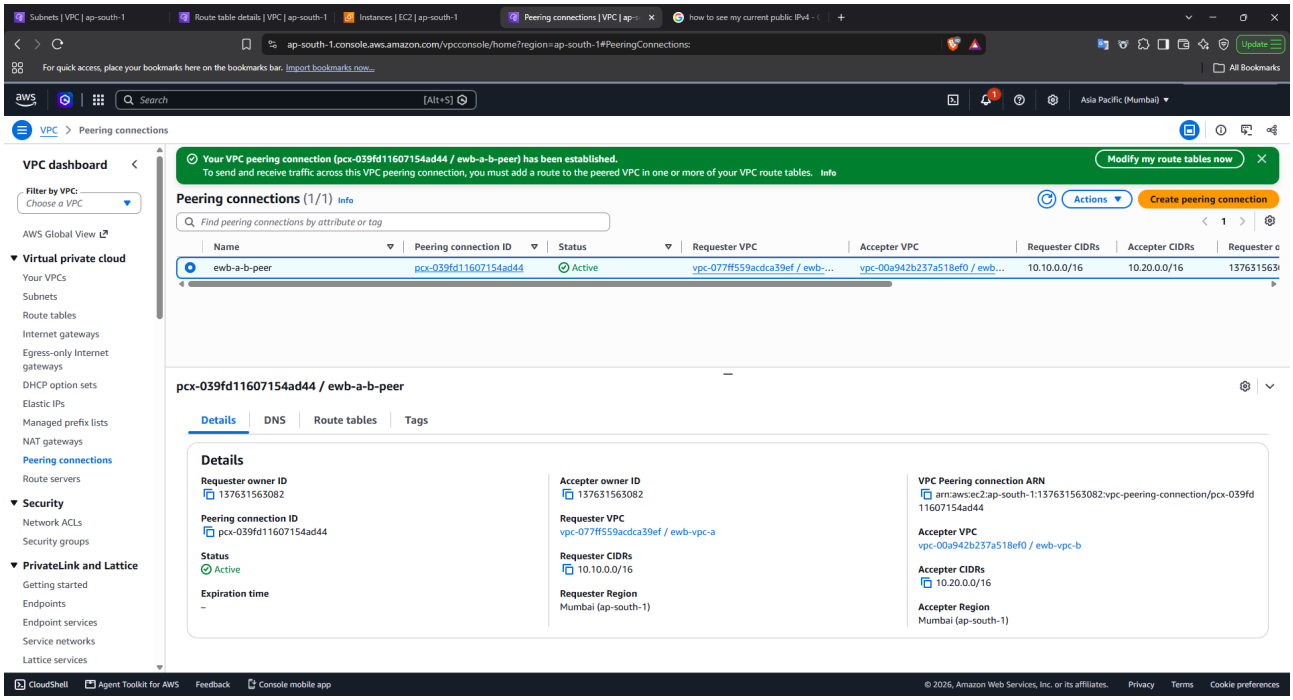


Figure 16 - Peering request created: status Pending acceptance, requester 10.10.0.0/16, accepter CIDR not yet shown.



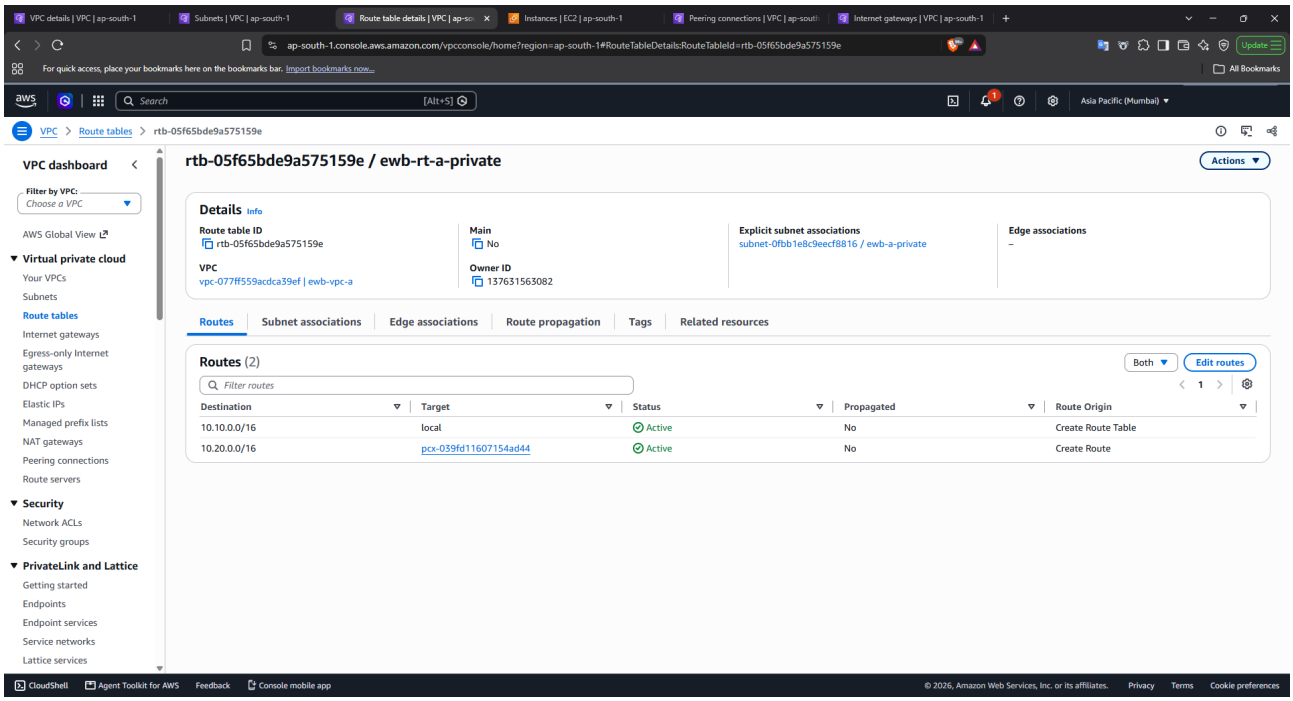
Checkpoint 03 - Figure 17 - After acceptance: status Active, pcx-039fd11607154ad44, requester ewb-vpc-a (10.10.0.0/16), accepter ewb-vpc-b (10.20.0.0/16).

8. Routes after peering

Route table	Destination	Target	Purpose
ewb-rt-a-private	10.20.0.0/16	pcx-039fd11607154ad44	Forward path App-A to VPC B
ewb-rt-b-private	10.10.0.0/16	pcx-039fd11607154ad44	Return path to VPC A

Route table	Destination	Target	Purpose
ewb-rt-a-public	10.20.0.0/16	pcx-039fd11607154ad44	Bastion's SSH connection to App-B

The public-table route is the one that is easy to forget: ProxyJump makes the bastion open the connection to App-B, so the bastion's subnet needs a route to VPC B. The more specific 10.20.0.0/16 route wins over 0.0.0.0/0. It is confirmed functionally by Figure 21, where ssh ewb-app-b works through the bastion.



Checkpoint 04 - Figure 18 - ewb-rt-a-private: 10.20.0.0/16 to the pcx, status Active.

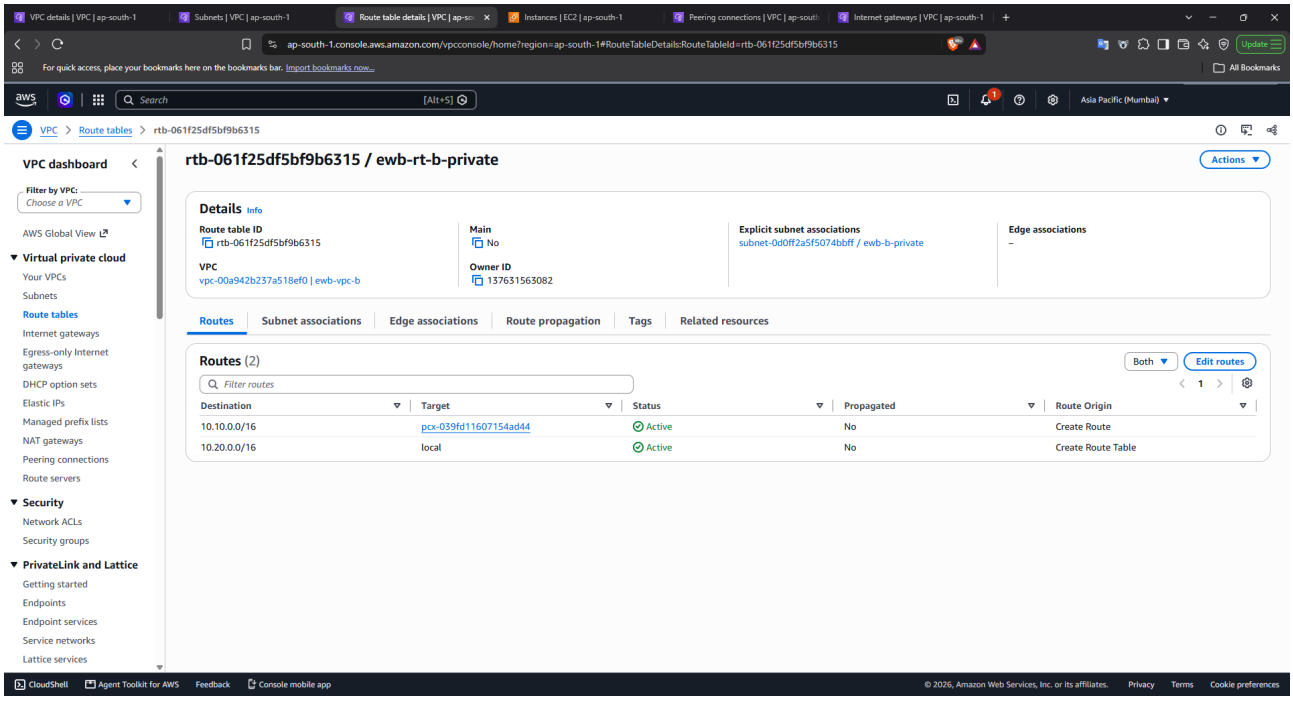
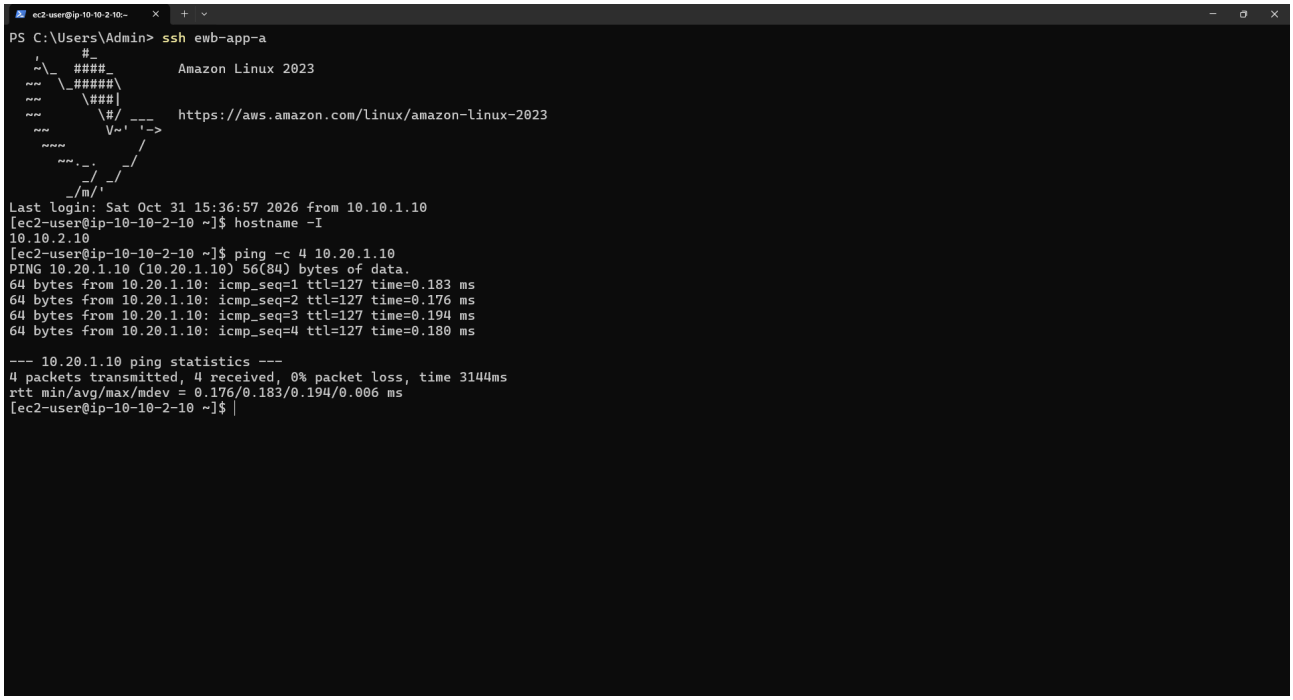
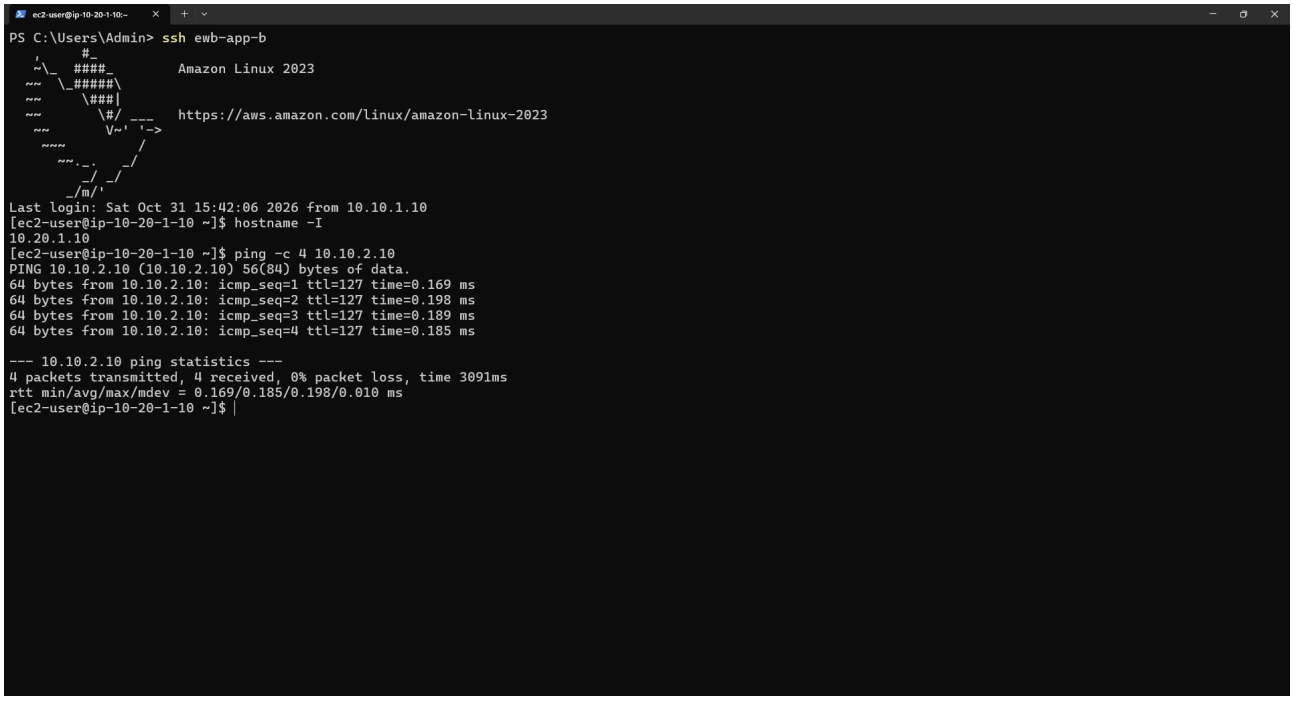


Figure 19 - ewb-rt-b-private: 10.10.0.0/16 to the pcx, status Active (return route).

9. Validation after peering



Test 02 - Figure 20 - Forward path from ip-10-10-2-10: 4 transmitted, 4 received, 0% packet loss to 10.20.1.10. Taken while the ICMP rules allowed the specific /32 source using the All ICMP type; see section 10 for the later correction to Echo Request.



Tests 03 and 04 - Figure 21 - ssh ewb-app-b from the laptop reaches App-B through the bastion (Last login source 10.10.1.10). hostname -I returns 10.20.1.10 and ping to 10.10.2.10 gives 0% loss. Same rule-type caveat as Figure 20.

Test 05 (isolation) is shown in Figures 10 and 11: neither private instance has a public IPv4 address.

10. Troubleshooting: wrong ICMP type in the security groups

After tightening the security groups I selected Custom ICMP with the protocol Echo Reply instead of Echo Request. ping sends Echo Request (type 8); Echo Reply (type 0) is the answer, and stateful security groups allow it automatically. With the wrong type, the destination dropped the pings even though routing and peering were correct.

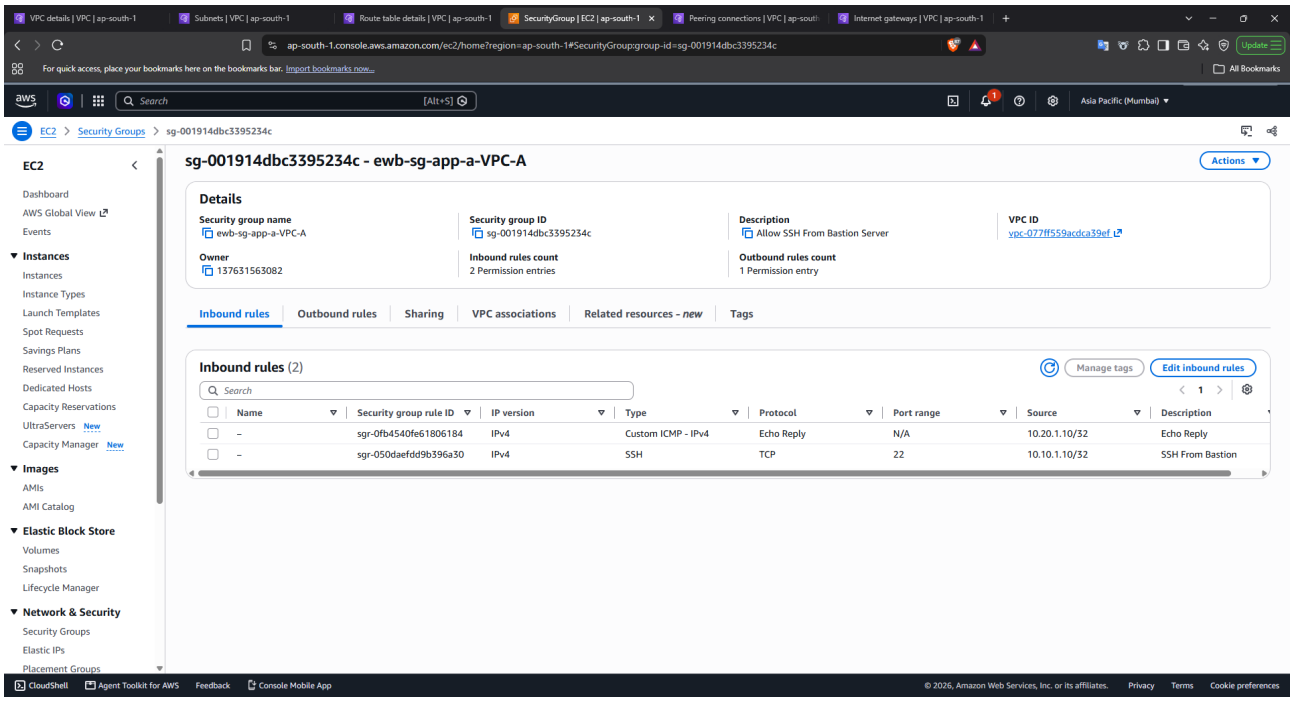


Figure 22 - ewb-sg-app-a-VPC-A with the ICMP rule's Protocol column showing Echo Reply (the incorrect type).

Symptom (real terminal output, App-A to App-B, then App-B to App-A):

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3122ms

[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
--- 10.10.2.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3096ms
```

Diagnosis: routes were Active in both directions and SSH worked, so the fault was the destination security group rule type. The console did not let me pick Echo Request, so I added the rule with AWS CloudShell (FromPort=8 is ICMP type 8):

```
aws ec2 authorize-security-group-ingress --region ap-south-1 --group-id sg-001914dbc3395234c \
  --ip-permissions IpProtocol=icmp,FromPort=8,ToPort=-1,\
  IpRanges='[{CidrIp=10.20.1.10/32,Description="Echo Request from App-B"}]'

Return: true   SecurityGroupRuleId: sgr-0b25b8bd8b5b49d39   FromPort: 8   CidrIpv4: 10.20.1.10/32
```

Result: the reverse ping from App-B then succeeded (real output):

```
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.187 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.194 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.187 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.201 ms
4 packets transmitted, 4 received, 0% packet loss, time 3091ms
```

11. Controlled failure

To show that the security group alone controls the ping, I removed only the ICMP rule (source 10.10.2.10/32) from ewb-sg-app-b-VPC-B and kept its SSH rule. Routes and the peering connection were not touched.

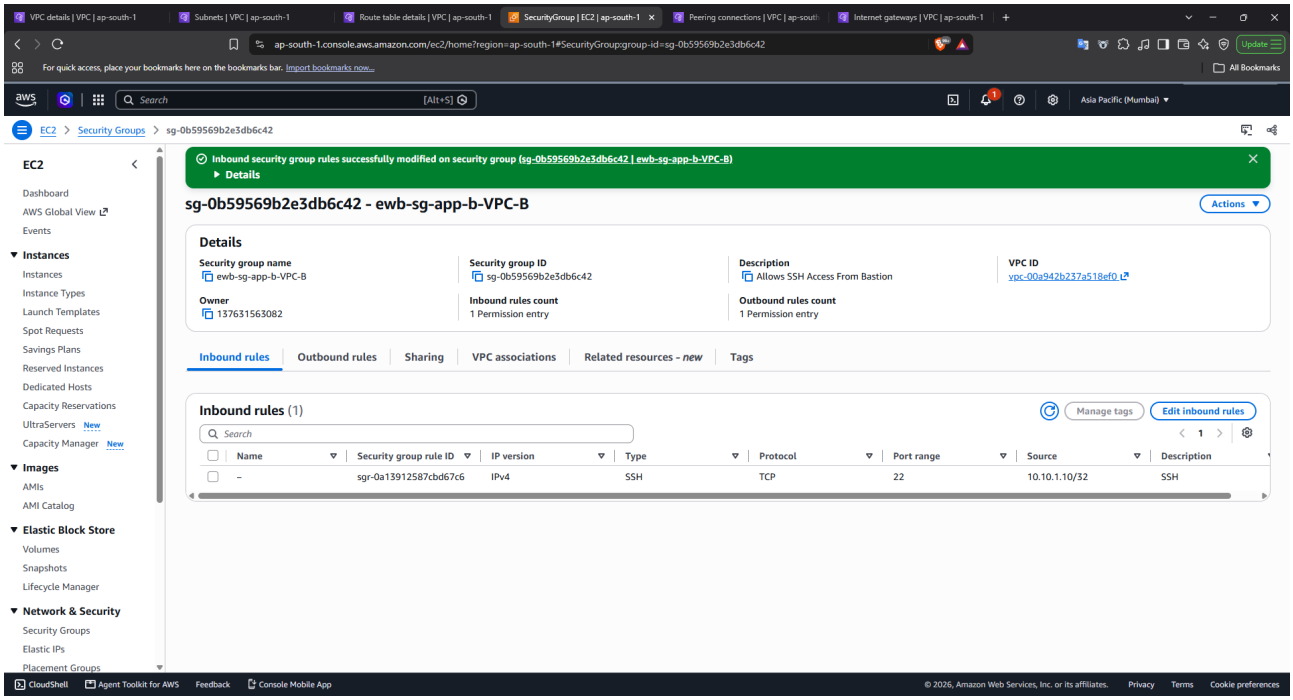


Figure 23 - ewb-sg-app-b-VPC-B after removal: Inbound rules (1), only SSH TCP 22 from 10.10.1.10/32. The console confirms the rules were modified successfully.

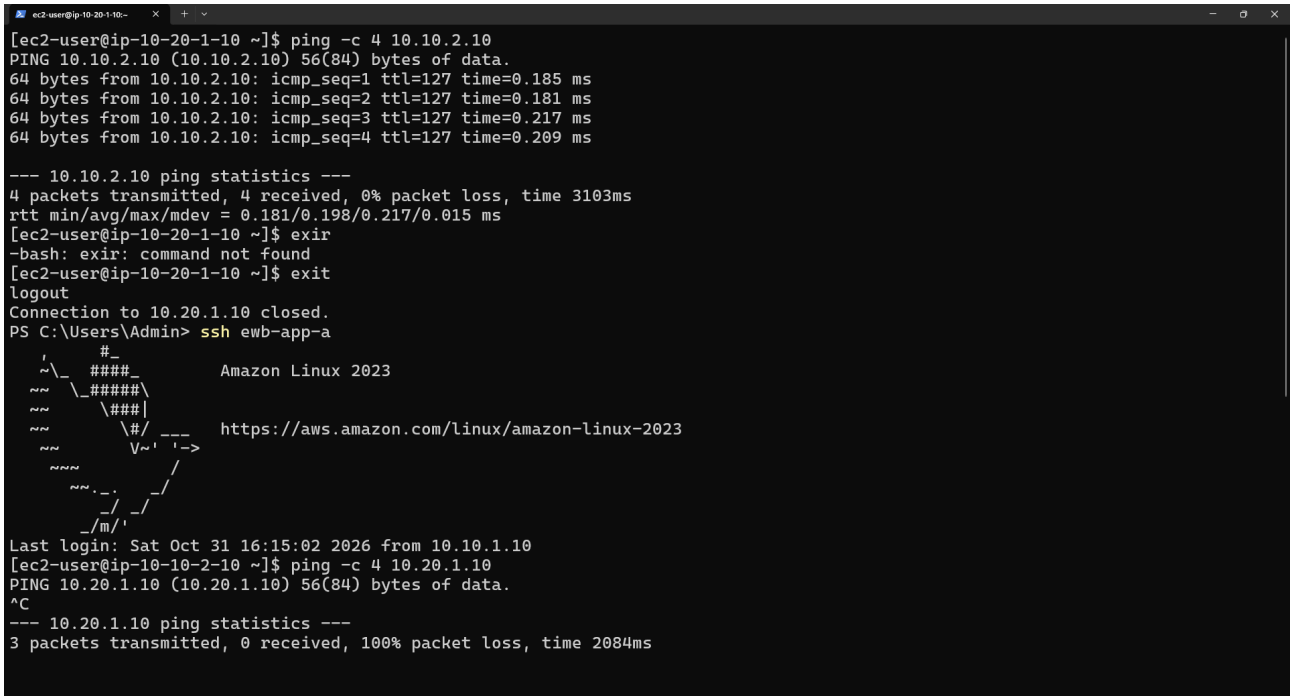


Figure 24 - Top: a successful reverse ping (0% loss) before the change. Bottom: new session on ip-10-10-2-10, ping -c 4 10.20.1.10 gets no replies (interrupted after 3 packets, 3 transmitted, 0 received, 100% loss).

Explanation: packets from App-A still crossed the peering connection, because the routes and the peering were unchanged. The destination security group then dropped the Echo Requests, so no replies came back. Only the security group rule changed, which shows that routing and security rules are independent checks and both must allow traffic.

12. Forward and return path

Forward (App-A to App-B): App-A (10.10.2.10) looks up 10.20.1.10 in ewb-rt-a-private, matches 10.20.0.0/16 to the pcx, and the packet crosses the peering connection into VPC B. ewb-sg-app-b must allow the request.

Return (App-B to App-A): App-B looks up 10.10.2.10 in ewb-rt-b-private, matches 10.10.0.0/16 to the pcx, and the reply crosses back. Replies are allowed by the stateful security group, but the route back must exist or the reply has nowhere to go.

13. Viva questions

1. Why is the return route needed? Peering does not create routes. Without 10.10.0.0/16 to the pcx in VPC B's route table, App-B can receive requests but cannot send replies, so the connection appears broken.

2. Why is the App-B SSH source the bastion's private IP? With ProxyJump the bastion opens a new SSH connection to App-B, so App-B sees 10.10.1.10 as the source, not my laptop.

3. Why does Active peering not guarantee a successful ping? Active only means the link exists. Packets also need routes in both VPCs and security group rules (and NACLs) that allow them. My Echo Reply mistake showed this: routes and peering were correct, but the ping still failed.

4. Can VPC B use VPC A's internet gateway over peering? No. Peering is non-transitive and does not forward traffic to an internet gateway on the other side. VPC B has no internet access.

14. What I learned

A VPC peering connection is only the link. Nothing flows until both sides have routes and the destination security group allows the traffic. Even with Active peering my ping failed until the routes existed in both directions, because replies need a path back just as requests do. The bastion's own subnet also needs a route to VPC B, since ProxyJump makes the bastion open the connection to App-B.

Several mistakes taught me more than the steps that worked. My bastion launched without a public IP because I did not enable it on the launch network interface. SSH to App-A hung because its rules were split across two security groups, and security groups drop unmatched traffic silently. On Windows, SSH ignored my config because the file was saved as config.txt. Finally, the ICMP rule had the wrong type (Echo Reply instead of Echo Request). Each failure pushed me to check the layers in order: instance, IP, route table, then security group.

The controlled failure showed how to isolate a fault: removing only the ICMP rule stopped the pings while routing and peering stayed intact. Keeping the private instances without public IPs is what keeps them isolated.